

AI-ASSISTED DEVELOPMENT

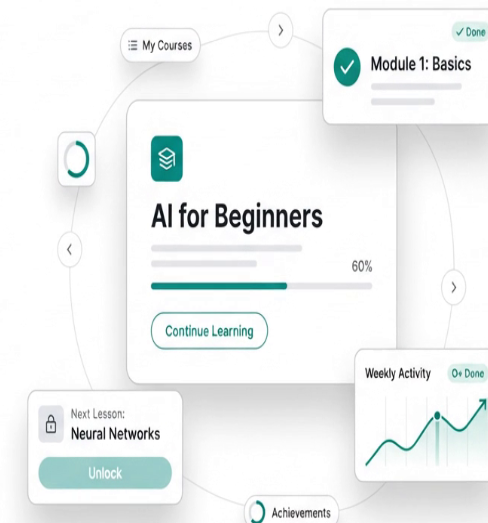
Vibe Coding untuk Membuat LMS Sederhana

Pendekatan praktis membangun aplikasi dengan AI: mulai dari intent yang jelas, iterasi cepat, sampai produk kecil yang benar-benar bisa digunakan.

Practical

AI + Software Development

Learning Management System



TOOLS YANG DIPERLUKAN

Untuk sesi ini, cukup ChatGPT + Codex.

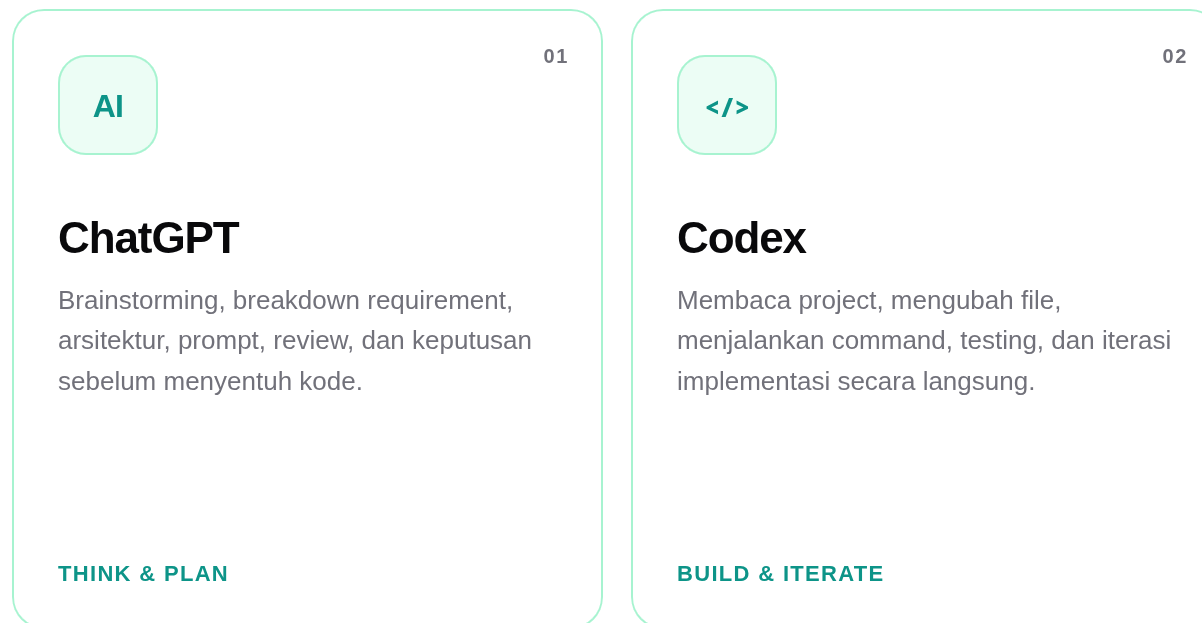
Vibe coding bisa dilakukan dengan banyak AI coding tools. Di materi ini kita fokus pada kombinasi ChatGPT untuk berpikir dan merancang, lalu Codex untuk mengeksekusi perubahan langsung ke codebase.

ALTERNATIF

Claude Code

Antigravity

Cursor



Intent → Implementation

MODEL ORCHESTRATION

Model berbeda, perannya juga berbeda.

Model dipisahkan berdasarkan peran: **perencanaan dan R&D** menggunakan reasoning yang lebih dalam, sedangkan **eksekusi** memprioritaskan kemampuan coding, tool use, dan iterasi yang cepat.

Fokus pada karakter workload, bukan hanya nama model.
Jika model yang sama tidak tersedia, gunakan model lain berdasarkan *role tier*: frontier reasoning untuk planner, dan strong agentic coding untuk executor.

Model roles yang digunakan

ROLE-BASED ROUTING

SOL

GPT-5.6 Sol
Planner, architecture, R&D, research synthesis, review keputusan besar.

HIGH

LUNA

GPT-5.6 Luna
Executor utama: implementasi, terminal/tool use, refactor, test, dan iterative coding.

X-HIGH

ROUGH ROLE EQUIVALENT

BUKAN BENCHMARK 1:1

Planner / R&D	≈	Claude Opus 5 · Gemini Pro-class
Executor / Coding agent	≈	Claude Sonnet 5 · Gemini Flash high-reasoning class

Yang dicari adalah karakter workload: reasoning depth, autonomy, tool use, latency, dan cost. Nama/tier vendor dapat berubah.

UI/UX FIRST

Sebelum bikin aplikasi, kunci dulu pengalaman dan tampilannya.

Di tahap ini kita belum membangun LMS production. PRD menjadi source of truth, lalu agent wajib membuat **DESIGN.md** sebagai spesifikasi UI/UX sebelum menyentuh prototype HTML. Urutannya: requirement → design spec → prototype.

- 01

PRD mengunci requirement
Scope, user journey, priorities, constraints, dan acceptance criteria.
- 02

Generate DESIGN.md
IA, screen specs, design tokens, components, states, responsive rules, dan accessibility.
- 03

Baru implement index.html
Prototype harus mengikuti DESIGN.md; jika design berubah, update DESIGN.md dulu.



PRD.md

Source of truth · LMS Sederhana

PRD.md

requirements

DESIGN.md

design spec

index.html

prototype

Review gate

approve

Buka PRD.md

PROTOTYPE FIRST

Fix UI/UX HTML sampai matang dulu.

Setelah PRD dan DESIGN.md selesai, generate prototype static HTML lalu fokuskan sesi berikutnya untuk memperbaiki pengalaman pengguna dan tampilannya. Jangan buru-buru masuk framework, backend, atau production logic.

Agentation membantu feedback lebih presisi.

Minta coding agent setup Agentation atau annotation inspector pada prototype agar kita bisa klik elemen, tulis revisi, lalu kirim selector + konteks UI langsung ke agent.

01

Generate prototype

Implement PRD.md + DESIGN.md menjadi single-file static HTML.

02

Setup Agentation

Tambahkan annotation workflow supaya feedback UI bisa menunjuk elemen yang tepat.

03

Inspect & annotate

Review hierarchy, spacing, alignment, content density, states, dan responsive behavior.

04

Fix & review lagi

Iterasi sampai UI/UX disetujui sebelum engineering production dimulai.

Generate → Inspect → Annotate → Fix → Review

ENTER THE PROJECT

Kasih agent sistem untuk belajar dari project-nya sendiri.

Sebelum coding lebih jauh, taruh satu bootstrap guide di project. Agent membacanya lalu membuat **AGENTS.md** dan workspace **.agents/** sebagai persistent self-improvement layer.

Planning + execution orchestration.

Request besar dipecah menjadi **master plan + child plans**. Saat diminta *implement / execute / eksekusi plan*, main agent hanya menjadi coordinator dan seluruh implementasi wajib didelegasikan ke sub-agent.

Buka AGENT_BOOTSTRAP.md

Project-local agent workspace

AUTO SELF-IMPROVEMENT

Folder di bawah adalah **baseline**, bukan whitelist. Agent boleh menambah folder lain di **.agents/** kalau memang ada kebutuhan reusable yang jelas.



.agents/memories/

Durable decisions, corrections, project conventions

REMEMBER



.agents/plans/

Master plan → child plans · dependencies · acceptance · evidence

PLAN



.agents/skills/

Reusable procedures and project-specific workflows

REUSE



.agents/knowledges/

Verified external docs, research, domain findings

LEARN



.agents/notes/

Follow-ups, review notes, temporary useful context

TRACK

Work



Validate



Reflect



Persist



Improve

Execute plan: coordinator → delegated sub-agents → parallel worktrees / sequential dependencies → validate + commit each completed unit → integrate → cleanup.

Tidak ada implementasi langsung di **main/master**. Worktree dipakai untuk workstream independen; dependency yang saling mengunci tetap dijalankan berurutan.

BOOTSTRAP ENGINEERING PROJECT

Ubah prototype ke stack nyata. Lalu STOP.

Mulai dari tahap ini, **single-file HTML** bukan lagi **delivery target**. Agent wajib membaca Tech Stack Contract, inspect repository, lalu bootstrap project ke struktur/framework yang sesuai stack tersebut.

Bootstrap only — jangan implement feature.

Fase ini hanya menyiapkan struktur project, architecture, AGENTS.md, implementation plan di .agents/, dan local guard. Jangan execute plan dan jangan menjalankan full product E2E.

Buka **ENGINEERING_GUARD.md**

Project bootstrap + engineering guard

HANDOFF BEFORE IMPLEMENTATION

Tech Stack Gate OS/target · Frontend · Backend/Server · Database · Observability

MISSING → STOP

FRONTEND DEPENDENCY DIRECTION

UI

UI → Components → Layout → Pages

BACKEND DEPENDENCY DIRECTION

API

Repositories → Use cases → Handler / Router

BOOTSTRAP OUTPUT

Project Bootstrap

Framework, config, dependencies, folders, entry points, and assets follow the declared tech stack — not the old one-file prototype.

Architecture & Guard

Clean Architecture, DRY, SOLID, KISS, YAGNI, strict lint/type/build, local pre-commit guard, no bypass.

Plan & AGENTS

Create/reconcile AGENTS.md and concrete implementation plans under .agents/. Planning only; do not execute them yet.

Hard Stop

No feature implementation and no full Playwright product E2E. Bootstrap verification only; page 8 starts execution.

Declared stack

bootstrap + plan + guard

STOP

Best practice

DRY

SOLID

KISS

YAGNI

Clean Architecture

.agents updated

— START IMPLEMENTATION

Sekarang jalankan implementasi melalui Codex berdasarkan plan yang sudah disiapkan.

PROMPT

CODEX

Read AGENTS.md, understand and follow all repository rules, then implement the existing plan.

Repository rules → Existing plan → Implementation

LET CODEX RUN

Implementasinya bisa lama. Nggak perlu ditungguin.

Codex akan jalan melewati seluruh plan, dependency, validation, dan commit workflow sampai semuanya selesai. Proses ini bisa makan waktu cukup panjang.

Tinggal jalanin di background. Sambil agent menyelesaikan semua plan, kita bisa lanjut dulu ke cara kolaborasi dengan AI tanpa harus menunggu implementasi selesai.

WHAT HAPPENS IN THE BACKGROUND

01

Read plans

Resolve dependencies dan urutan eksekusi.

PLAN

02

Implement

Kerjakan workstream sesuai aturan repository.

CODE

03

Validate + commit

Guard, tests, review, lalu lanjut plan berikutnya.

REPEAT

04

Finish all plans

Berhenti setelah seluruh execution plan terimplementasi.

DONE

Sambil nunggu...[LANJUT: CARA KOLABORASI →](#)

CARA KOLABORASI

Satu developer pegang satu fitur dari depan sampai belakang.

Jangan pecah ownership jadi tim frontend versus backend untuk fitur yang sama. Satu developer atau agent memegang satu **vertical slice**: UI, API, business logic, data, dan test sampai fiturnya benar-benar selesai.

Kenapa? Context tetap utuh, handoff berkurang, ownership jelas, dan risiko dua orang saling menunggu kontrak frontend/backend jauh lebih kecil.

One feature = one accountable owner

VERTICAL SLICE

01

UI / interaction

User flow, states, validation, accessibility.

FRONTEND

02

API / application boundary

Request contract, authorization, error behavior.

SERVER

03

Domain + persistence

Business rule, repository, migration/data impact.

LOGIC

04

Tests + observability

E2E, integration, logs/traces, acceptance evidence.

DONE

Feature intent



One owner



End-to-end delivery

EXCEPTION

Kalau consume service di luar proyek, batas ownership-nya berubah.

Developer fitur tetap mengerjakan vertical slice di proyek kita, tetapi **tidak mengambil ownership internal service eksternal**. Yang kita miliki adalah integration boundary: adapter/client, contract, resilience, security, dan observability.

Kalau service eksternal juga sedang berubah: koordinasi lewat versioned API contract/schema dan sandbox/mock—bukan dengan mencampur branch atau source code dua repository.

Project boundary contract

INTEGRATION FIRST

01

Contract

OpenAPI/webhook schema, versioning, expected errors.

AGREE

02

Adapter

Keep vendor/service details behind one internal interface.

ISOLATE

03

Resilience

Timeout, retry policy, idempotency, graceful failure.

PROTECT

04

Security + telemetry

Secrets, auth, correlation ID, logs/traces/metrics.

OBSERVE

Our feature



Adapter / contract



External service

BRANCHING

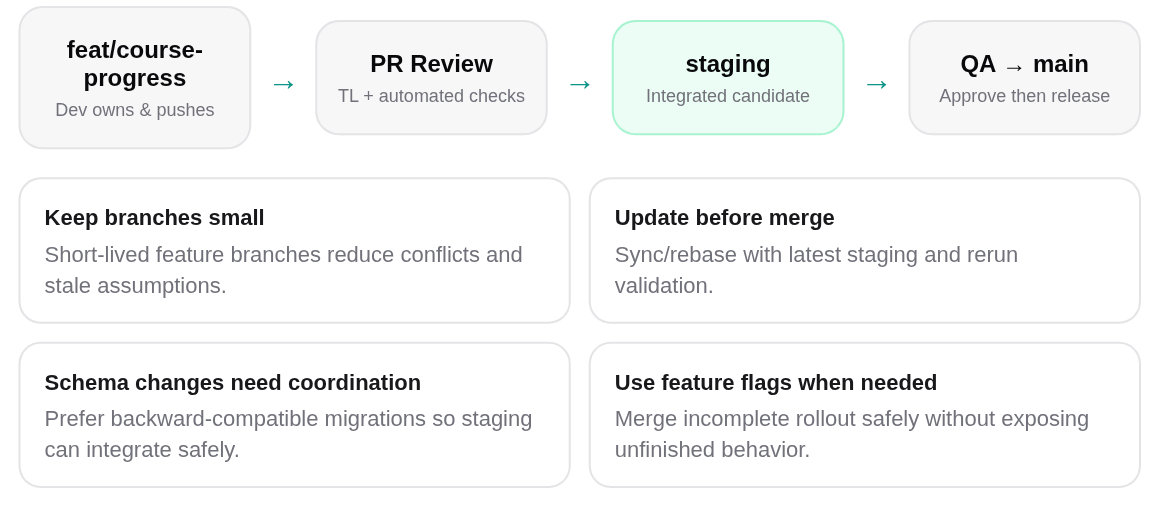
Satu fitur wajib punya satu branch sendiri.

Developer cukup bekerja dan push ke branch fiturnya. **Tech Lead yang menggabungkan ke staging**, lalu QA menguji hasil integrasi sebelum perubahan boleh masuk ke main/production.

Yang sering kelewat: feature branch harus short-lived, staging/main protected, PR wajib review + status checks, dan jangan force-push ke shared branches.

Feature → Staging → QA → Production

CONTROLLED MERGE



RELEASE

Tag release supaya versi yang sudah jalan bisa dilacak dan di-rollback.

Gunakan GitHub Releases + immutable Git tag untuk setiap versi yang benar-benar dilepas. Simpan changelog dan artifact/build identity agar tim tahu persis source mana yang menghasilkan versi tersebut.

Web: rollback biasanya lebih cepat karena server bisa redeploy versi sebelumnya. Tapi tetap penting saat ada migration, config, cache, atau perubahan API yang membuat rollback tidak sesederhana mengganti commit.

Why native makes releases critical

ROLLBACK REALITY

Web deployment

Tim mengontrol server dan biasanya bisa redeploy known-good build dalam hitungan menit.

Native app

Binary sudah tersebar ke device pengguna. Kita tidak bisa menarik dan menggantinya seketika.

Store delay

App Store/Play Store review dan rollout membuat hotfix tidak selalu langsung sampai ke pengguna.

Multiple versions live

Backend harus tetap kompatibel karena pengguna bisa menjalankan beberapa versi aplikasi sekaligus.

Release checklist yang jangan kelewat: semantic/versioned tag, changelog, signed/native artifacts bila relevan, migration compatibility, phased rollout, monitoring, dan known-good rollback target.

READY TO DEPLOY

Coding selesai belum berarti siap production.

Sebelum deploy, pastikan artifact yang lolos QA memang artifact yang sama yang dipromosikan ke production.

Build once.

QA the artifact.

Promote the same artifact.

Pre-deployment gate

MUST PASS

01

Environment & secrets

Config dev/staging/prod jelas; secret tidak masuk Git.

02

Database migration

Versioned, tested, dan punya rollback/forward-fix plan.

03

Observability

Logs, metrics, traces, health/readiness, alerting aktif.

04

QA / UAT sign-off

Commit SHA, tag, image, atau binary bisa ditelusuri.

05

Security checks

Dependency, permission, dan exposed config sudah dicek.

06

Rollback trigger

Tentukan threshold error, crash, latency, atau critical-flow failure.

Feature → Staging → QA → Release → Deploy → Observe

CARA DEPLOYMENT

Promote release yang sudah lolos QA. Jangan bikin versi baru saat deploy.

Deployment adalah proses memindahkan **artifact yang sama** dari staging ke production, menjalankan perubahan yang dibutuhkan, memastikan service sehat, lalu mengawasi dampaknya.

Prinsip: release harus traceable ke commit SHA/tag tertentu. Kalau ada masalah, kita harus tahu persis versi mana yang sedang berjalan dan versi mana yang menjadi rollback target.

Production promotion flow

CONTROLLED ROLLOUT

01

Select approved release

Artifact/tag yang sudah lolos staging + QA.

RELEASE

02

Apply production config

Environment, secrets, network, database target.

CONFIG

03

Deploy safely

Canary, rolling, blue/green, atau controlled rollout.

SHIP

04

Smoke + health checks

Critical flow, readiness, dependency connectivity.

VERIFY

05

Observe or rollback

Error rate, latency, logs, traces, business signals.

WATCH

Release → Deploy → Verify → Observe → Promote / Rollback

CARA DEPLOYMENT

Tiga cara paling gampang buat mulai deploy server.

Untuk demo, pilih sesuai bentuk aplikasinya. Yang penting prosesnya bisa **start**, **restart**, **lihat log**, dan **recover** tanpa menjalankan app manual dari terminal.

Di depan process manager:

Nginx / Caddy → reverse proxy → localhost:3000

Domain + TLS/HTTPS

systemd

LINUX SERVICE

Cocok untuk binary/service apa pun: Go, Node, Python, atau executable lain.

```
sudo systemctl enable --now sensio
sudo systemctl status sensio
journalctl -u sensio -f
sudo systemctl restart sensio
```

Native ke OS · auto-start saat boot · log via journald

PM2

NODE.JS

Praktis untuk Next.js/Node kalau mau process manager tanpa container.

```
pm2 start "pnpm start" --name sensio
pm2 save
pm2 logs sensio
pm2 restart sensio
```

Simple untuk Node · restart/cluster/log management

Docker

PORTABLE

Environment lebih konsisten dan artifact deployment lebih mudah dipindah.

```
docker build -t sensio:1.0.0 .
docker run -d --name sensio \
  -p 3000:3000 --env-file .env.production \
  sensio:1.0.0
docker logs -f sensio
```

Build image sekali · version/tag jelas · mudah rollback image

Build → Start → Reverse proxy → Health check → Logs → Restart / rollback

NATIVE APP

Native bukan sekadar deploy. Kita mendistribusikan binary ke device yang tidak kita kontrol.

APK/AAB/IPA harus ditandatangani, diuji sebagai artifact final, lalu masuk internal/beta track sebelum production rollout. Setelah binary tersebar, user dapat berada di banyak versi sekaligus.

Rollback native tidak instan. Kita tidak bisa menarik binary yang sudah terpasang seperti mengganti container server. Biasanya recovery dilakukan lewat feature flag, server-side mitigation, atau merilis binary baru.

Native release path

DISTRIBUTION, NOT JUST DEPLOY

- 01 Build + sign immutable binary**
Version/build number, signing key, reproducible artifact. **BINARY**
- 02 Internal / beta distribution**
TestFlight, Play internal/closed track, enterprise channel. **BETA**
- 03 Store review + approval**
Release timing bisa bergantung proses eksternal. **STORE**
- 04 Phased rollout**
Mulai persentase kecil, monitor crash dan critical flow. **ROLLOUT**

Backend compatibility

API harus tetap melayani versi app lama selama masa transisi.

Feature flags

Matikan fitur bermasalah tanpa menunggu review binary baru.

Crash monitoring

Crash-free users/session menjadi sinyal rollout penting.

Emergency release

Siapkan hotfix path karena rollback penuh sering tidak tersedia.

SETELAH DEPLOY

Deploy sukses belum berarti aplikasi sehat.

Begitu versi baru menerima traffic, lakukan smoke test pada jalur paling penting dan pastikan sistem pendukungnya benar-benar hidup.

Rule:

Verifikasi menggunakan production behavior yang nyata, bukan hanya status process atau container.

Post-Deployment Verification

PRODUCTION OPERATIONS

01

Health & readiness

Process hidup, instance ready, dependency reachable.

02

Critical user flow

Login, transaksi utama, lesson/course flow, atau fungsi bisnis inti.

03

Data & workers

DB/cache/storage terhubung; queue, cron, worker, webhook berjalan.

04

Observe signals

Error rate, latency, saturation, logs, metrics, traces tidak menyimpang.

05

Version evidence

Commit/tag/artifact yang aktif sama dengan release yang disetujui.

06

Smoke after change

Ulangi setelah migration, scaling, config, atau traffic switch.

Deploy



Smoke



Observe



Confirm

KALAU PRODUCTION RUSAK

Stabilkan user dulu. Investigasi bisa menyusul.

Kalau critical flow rusak atau indikator produksi melewati batas aman, jangan terpancing memperbaiki langsung di production bila rollback lebih cepat dan aman.

Rollback trigger:

Critical journey gagal, error spike, crash, severe latency, data-integrity risk, atau dependency failure yang tidak bisa ditoleransi.

Incident & Rollback

PRODUCTION OPERATIONS

01

Detect

Alert/monitoring menunjukkan regression atau outage.

02

Decide quickly

Bandingkan risiko rollback versus hotfix/roll-forward.

03

Rollback first

Promote known-good artifact bila itu jalur recovery paling aman.

04

Protect data

Pastikan migration/data change tidak membuat rollback destruktif.

05

Communicate

Catat impact, owner, status, dan keputusan recovery.

06

Postmortem

Setelah stabil: root cause, correction, prevention, runbook update.

Signal



Decide



Recover



Verify



Learn

OPERATIONAL SAFETY

Backup yang belum pernah direstore belum terbukti berguna.

Backup bukan sekadar file tersimpan. Tentukan apa yang dibackup, retention, recovery target, akses, enkripsi, lalu latihan restore secara berkala.

Sebelum perubahan berisiko:

Ambil backup/snapshot sebelum destructive migration atau operasi data besar, tetapi tetap desain migration supaya aman dan reversible bila memungkinkan.

Backup & Restore

PRODUCTION OPERATIONS

01

Scope

Database, object storage, config penting, keys sesuai policy.

02

Retention

Daily/weekly/monthly sesuai kebutuhan dan biaya.

03

Off-site / isolation

Jangan simpan satu-satunya backup di failure domain yang sama.

04

Restore test

Uji restore ke environment terisolasi dan verifikasi integritas.

05

RPO / RTO

Tahu berapa data yang boleh hilang dan berapa lama recovery.

06

Access & audit

Batasi akses backup, encrypt, log siapa yang restore/download.

Backup



Verify



Restore Test



Ready

BONUS: MOTION / 3D RESEARCH → GOOGLE FLOW ASSETS

Bikin landing page terasa hidup tanpa menunggu aset final.

Agent mulai dengan **full resource gathering** untuk **motion, 3D, interactive, dan cinematic website** dari berbagai sumber, lalu memilih direction yang paling cocok dengan produk sebelum menyelaraskan PRD, DESIGN, implementasi, dan asset Google Flow.

Template adalah reference, bukan penjara.

Agent boleh update PRD.md, DESIGN.md, struktur section, hingga rombak index.html bila perlu agar hasil akhir benar-benar cocok dengan produk dan direction motion / 3D yang dipilih.

Buka prompt guide

Cinematic reference workflow

RESEARCH → STAGE → ANIMATE

Product → Cinematic research → 3D / camera direction → Dummy final slot → Google Flow

Reference dipilih dari fit, bukan sumber
Asset mengikuti template direction + style HTML aktual. Background, crop, scale, radius, dan framing dipetakan berdasarkan component yang benar-benar dipakai.

Video	image-first	HTML mapping	CSS
I01_hero-base.md	approve still	object-fit	crop
V01_from-image.md	animate	object-position	focus
assets/hero-loop.mp4	replace	background	match HTML

Full resource gathering → pilih motion / 3D direction → update PRD/DESIGN/index bila perlu → generate image → animate dari image → map scale/crop di HTML → replace asset → validate.

VIBE CODING LMS

Terima kasih.

Semoga workflow ini bisa langsung dipakai untuk membangun, berkolaborasi, dan membawa produk dari ide sampai production dengan lebih terstruktur.

DESIGN FOUNDATION

DESIGN GUIDE

Fondasi visual deck sebelum kita masuk ke materi utama.

[Design principles](#)[Visual system](#)[PowerPoint-safe layout](#)

DESIGN PRINCIPLES

Sensio-native: minimal, precise, dan product-first.

Deck mengikuti bahasa visual Sensio: clean surfaces, hierarchy tegas, accent seperlunya, dan komponen yang terasa seperti bagian dari satu product system.

01

Product Clarity

Satu pesan utama per slide. Informasi disusun seperti UI Sensio: cepat dipindai, tegas, dan bebas noise.

02

Quiet Confidence

Gunakan surface terang, near-black text, border lembut, dan whitespace yang cukup. Profesional tanpa terasa dingin atau dekoratif.

03

System over Styling

Warna, radius, border, shadow, dan spacing mengikuti token Sensio. Variasi visual harus lahir dari sistem, bukan improvisasi per slide.

04

Presentation-safe Product UI

Komposisi tetap 16:9 dan PowerPoint-safe, tetapi treatment visualnya menyerupai product surface Sensio: modular, clean, dan mudah dibaca.

STYLE GUIDE

Sensio visual system untuk semua slide.

Typography, palette, spacing, radius, border, dan shadow mengikuti implementasi Sensio agar deck dan produk terasa satu keluarga.

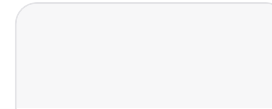
CORE PALETTE



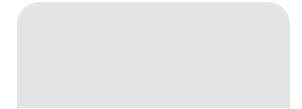
Foreground
#09090B



Primary
#0D9488



Surface
#F7F7F8



Border
#E4E4E7

TYPOGRAPHY

DISPLAY · 44–58 px

Vibe Coding

HEADING · 19–34 px

Structured visual hierarchy

BODY · 14–21 px

Inter sebagai visual reference Sensio; fallback Segoe UI / Arial tetap disiapkan untuk kompatibilitas PowerPoint.

LAYOUT RULES

16:9 · 1280×720 canvas · 64 px horizontal safe area · 48 px vertical safe area · maksimal 2 kolom utama · radius 8–16 px · border tipis · shadow sangat halus · satu focal point per slide.

VISUAL GUARDRAILS

Aturan yang tidak boleh dilanggar di slide berikutnya.

01

One message per slide

Setiap slide harus punya satu takeaway utama. Kalau ada dua ide besar, pecah jadi dua slide.

02

Maximum two columns

Gunakan satu kolom untuk narasi kuat atau dua kolom untuk perbandingan. Hindari layout 3–4 kolom kecuali berupa card ringkas.

03

Accent is intentional

Sensio brand #0D9488 hanya untuk emphasis, active state, key number, atau focal action. Jangan menjadikannya warna dekoratif massal.

04

No decorative noise

Hindari gradient dekoratif. Shadow harus tipis seperti Sensio; icon, ilustrasi, dan shape hanya dipakai bila memperjelas informasi atau hierarchy.

05

Whitespace is content

Jangan memenuhi canvas hanya karena masih ada ruang kosong. Ruang kosong menjaga fokus dan kualitas visual.

06

Consistency beats novelty

Jangan membuat gaya baru per slide. Gunakan pattern Sensio yang sama: clean surface, border subtle, radius 8–16 px, accent biru terkontrol, dan spacing konsisten.

LAYOUT SYSTEM

Grid dan spacing dibuat predictable.

Semua slide berikutnya harus terasa berasal dari sistem yang sama, walaupun jenis kontennya berbeda.

CANVAS

1280 × 720 / 16:9. Tidak ada elemen penting di luar safe area.

SAFE AREA

64 px horizontal · 48 px vertical. Header/footer tidak boleh dimakan konten utama.

SPACING

Gunakan kelipatan **8 px**. Jarak antarseksi 24–40 px; antarelemen kecil 8–16 px.

TEXT WIDTH

Paragraf panjang maksimal sekitar **680–720 px**. Hindari baris teks selebar slide.

ALIGNMENT

Default **left aligned**. Center hanya untuk title/divider atau pesan tunggal yang memang membutuhkan fokus.

CARDS

Radius **8–16 px**, border #E4E4E7, shadow sangat ringan. Card bukan dekorasi; hanya dipakai untuk grouping informasi.

DO

Bikin slide yang cepat dipahami.

- ✓ Judul menyampaikan kesimpulan atau intent, bukan label generik.
- ✓ 3–5 poin utama per slide; jika lebih, pecah menjadi beberapa slide.
- ✓ Body 14–21 px, heading 19–34 px, display 44–58 px.
- ✓ Gunakan diagram, screenshot, atau card hanya jika mempercepat pemahaman.
- ✓ Highlight hanya 1–2 elemen paling penting per slide.

DON'T

Hindari pola yang bikin deck terasa amateur.

- ✗ Paragraf panjang, font kecil, atau bullet sampai memenuhi seluruh canvas.
- ✗ Terlalu banyak warna, icon random, gradient dekoratif, atau shadow tebal yang tidak sesuai bahasa visual Sensio.
- ✗ Mengubah alignment, radius, warna utama, dan spacing hanya demi variasi.
- ✗ Menambahkan visual filler yang tidak membantu audience memahami pesan.
- ✗ Membuat slide berbeda total dari sistem hanya karena kontennya berbeda.